



UNIVERSITY OF
SURREY

A Gradient-Guided Evolutionary Neural Architecture Search

Xue, Yu; Han, Xiaolong Han; Neri, Ferrante; Qin, Jiafeng; Pelusi, Danilo

<https://openresearch.surrey.ac.uk/esploro/outputs/journalArticle/A-Gradient-Guided-Evolutionary-Neural-Architecture-Search/99863466602346/filesAndLinks?index=0>

Xue, Y., Han, X. H., Neri, F., Qin, J., & Pelusi, D. (2025). A Gradient-Guided Evolutionary Neural Architecture Search. *IEEE Transactions on Neural Networks and Learning Systems*, 36(3), 4345–4357.

<https://doi.org/10.1109/TNNLS.2024.3371432>

Document Version: Author's Accepted Manuscript

Published Version: <https://doi.org/10.1109/TNNLS.2024.3371432>

Surrey Open Research repository homepage: <https://openresearch.surrey.ac.uk/esploro/openresearch@surrey.ac.uk>

CC BY V4.0

Research:Open

© 2024 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works."

Downloaded On 2026/02/18 13:54:30 +0000

A Gradient-Guided Evolutionary Neural Architecture Search

Yu Xue, *Senior Member, IEEE*, Xiaolong Han, Ferrante Neri, *Senior Member, IEEE*, Jiafeng Qin, Danilo Pelusi

Abstract—Neural architecture search (NAS) is a popular method that can automatically design deep neural network structures. However, designing a neural network using NAS is computationally expensive. This paper proposes a gradient-guided evolutionary neural architecture search (GENAS) to design convolutional neural networks for image classification. GENAS is a hybrid algorithm that combines evolutionary global and local search operators to evolve a population of subnets sampled from a supernet. Each candidate architecture is encoded as a table describing which operations are associated with the edges between nodes signifying feature maps. Besides, evolutionary optimisation employs novel crossover and mutation operators to manipulate the subnets using the proposed tabular encoding. Every n generations, the candidate architectures undergo a local search inspired by differentiable neural architecture search. GENAS is designed to overcome the limitations of both evolutionary and gradient-descent NAS. This algorithmic structure enables the performance assessment of the candidate architecture without retraining, thus limiting the NAS calculation time. Furthermore, subnet individuals are decoupled during evaluation to prevent strong coupling of operations in the supernet.

The experimental results indicate that the searched structures achieve test errors of 2.45%, 16.86%, and 23.9% on CIFAR-10/100/ImageNet datasets and it costs only 0.26 GPU days on a graphic card. GENAS can effectively expedite the training and evaluation processes and obtain high-performance network structures.

Index Terms—Neural architecture search, evolutionary computation, gradient optimisation, image classification

I. INTRODUCTION

CONVOLUTIONAL neural networks [1], [2] are well-established computational models and essential components of machine learning. They have demonstrated their effectiveness across various domains, including image recognition [3], [4] and object detection [5]. The performance of neural networks depends significantly on two factors: network parameters and architecture. Currently, gradient

descent methods, such as stochastic gradient descent (SGD) and the Adam optimiser, are widely used to optimise network parameters. However, when it comes to network architectures, designing them manually, as seen in popular structures like VGG [6] and ResNet [7], can be an extensive and resource-intensive task, often requiring significant human effort and computational resources for the trial and error. This challenge becomes even more pronounced with deep neural networks.

As a solution to this problem, neural architecture search (NAS) has emerged as a field of interest for researchers and practitioners. NAS encompasses a set of methods aimed at automatically designing neural architectures to tackle one or more specific tasks [8]. Among the many, some recent successful studies on NAS are reported in [9]–[13]

Broadly following the categorisation proposed in [14], search strategies can be divided into three main streams: 1) reinforcement learning (RL); 2) gradient-based; 3) evolutionary computation. Reinforcement learning treats architecture search as the action of an agent and constructs the network through various behaviours. Subsequently, the network's performance is used as the reward. For instance, Zoph *et al.* [15] employed a recurrent neural network (RNN) strategy to sample strings and then encode neural architectures. Although promising, the reinforcement learning logic embedded in this class of methods demands a significant amount of computing resources. For example, Zoph *et al.* [15] utilised 800 GPUs and three weeks for searching for an optimal network structure.

In contrast to RL, gradient-based methods are faster at detecting promising architectures. These methods initially construct a supernet that defines the search space. Then, they apply a gradient-descent method to continuously optimise the architectural parameters of the network. The method converges to a subnet, which may be a local optimum of the search space. Algorithms belonging to this category can quickly identify a promising architecture. For example, Liu *et al.* [16] proposed a method, named differentiable architecture search (DARTS), which has proven to be able to design a trained promising architecture within 1.5 days with a single GPU machine. However, the application of DARTS, due to the designing of the search space and the sampling strategy of the candidate operations, is prone to get sub-optimal architectures containing multiple redundancies and/or a large number of skip connections [17], [18].

Moreover, DARTS is known to suffer from the so-called

This work was partially supported by the National Natural Science Foundation of China (62376127, 61876089, 61876185, and 61902281), the Natural Science Foundation of Jiangsu Province (BK20141005). (Yu Xue and Ferrante Neri are both corresponding authors.)

Yu Xue, Xiaolong Han, and Jiafeng Qin are with the School of Software, Nanjing University of Information Science and Technology, Jiangsu, China. E-mails: xueyu@nuist.edu.cn; xiaolonghan@nuist.edu.cn; qinjiafeng@nuist.edu.cn.

Ferrante Neri is with the Nature Inspired Computing and Engineering Research Group, Department of Computer Science, University of Surrey, Guildford, Surrey, United Kingdom. E-mail: f.neri@surrey.ac.uk.

Danilo Pelusi is with the Faculty of Communication Science, University of Teramo, Coste Sant'Agostino, Via Renato Balzarini, 1, 64100 Teramo TE, Italy. E-mail: dpelusi@unite.it.

coupling problem: the learned architecture’s weights are tightly coupled with the architecture itself [19]. More precisely, DARTS optimizes all architectural parameters simultaneously for each pair of nodes. Subsequently, the operation linked to the architectural parameter with the highest value is preserved, while all others are pruned. The study in [19] demonstrates that the architectural parameter (also known as operation weight) value signifies the importance of the operation on the specific edge, but does not provide evidence that the associated edge should be retained in the searched cell. The coupling between a high value of operation weight and the presence of the associated edge in the designed network is arbitrary and is likely to lead to a suboptimal design. On the other hand, the study in [20] also demonstrates that the tight coupling makes it challenging to separate the optimisation of the architecture (e.g., deciding which operations to use in each layer) from the optimisation of the weights (e.g., adjusting the parameters of those operations). The coupling problem can be problematic because it hinders the ability to search for optimal neural network architectures efficiently and can slow down the search process, making it computationally expensive.

Evolutionary NAS [21] (ENAS) can better explore the entire search space by encoding the network architecture and implementing selection, crossover, mutation, and other operations over a population of candidate architectures [22]. The global search nature of ENAS, unlike gradient-based methods, can avoid the search falling into a local optimum. On the other hand, ENAS usually requires a fairly large amount of computational resources since the evaluation of the performance (a.k.a. fitness) of each candidate network involves an independent training process. For example, Xie *et al.* [23] used a fixed-length binary string to encode the network structures. On the CIFAR-10 dataset, the algorithm required one day on 17 GPUs. Moreover, some network structures with low performance may be constructed due to random initialisation of the network models and, more generally, due to the stochasticity of the search space. Hence, a successful ENAS imposes the design of appropriate search space, constraints, and moving operators.

Taking inspiration from pioneering studies on memetic computing for CNN design [24], this paper introduces a novel approach that harnesses the strengths of both DARTS and ENAS methodologies. The proposed algorithm, named gradient-guided evolutionary architecture search (GENAS), employs a supernet to define the search space and then evolves a population of candidate architectures, each functioning as a subnet. During each generation, the offspring individuals, generated through crossover and mutation, directly inherit their weights from the supernet, eliminating the need for retraining their parameters. This mechanism significantly accelerates the evaluation process for the subnet individuals. The gradient-descent method from DARTS is applied after several generations to refine the architectures further. The use of evolutionary operators, which performs the selection of the operations between pairs of nodes based on the model

accuracy, alleviates the coupling problem caused by the DARTS-like gradient-based search. Furthermore, the second hard pruning operation (Section III-D), which selects the operation with the highest structural weight between each pair of nodes when evaluating the population, ensures that the search and test networks (Section IV-C) exhibit similar performance.

Our contributions are summarised as follows:

- 1) We propose a novel tabular encoding for the subnets. This encoding enables the manipulation of network architectures through both evolutionary operators and gradient-descent local search in a natural way.
- 2) We propose the use of an evolutionary framework on a DARTS search space. This hybridisation helps to overcome some of the limitations of both ENAS and gradient-based NAS. The DARTS search space is small and efficient, thus able to drastically reduce the calculation time required by the evolutionary optimisation. Moreover, the utilization of a supernet allows for the evaluation of candidate architectures’ quality without retraining. On the other hand, evolutionary operators compensate for the gradient-descent search, mitigating the risk of convergence into a local optimum. Furthermore, the proposed evolutionary operators on the tabular encoding perform a decoupled search, thus alleviating coupling problems introduced by DARTS. Lastly, the gradient-descent local search exploits the solutions detected by the evolutionary operators, producing quick improvements upon them.

We organised the rest of this paper as follows. In section II, we describe the search space and search method of NAS. In section III, we introduce a neural architecture search method (GENAS) that combines evolutionary computation and gradient descent. The section IV introduces the experimental method, supernet structure, dataset, parameters, and proves the performance of GENAS by comparing it with other NAS methods. Finally, the conclusion and future works are given in section V.

II. RELATED WORK

In NAS, the search space defines the neural architectures available for exploration, while the search strategy dictates the algorithm applied to efficiently and accurately discover the optimal network architecture. This section offers an overview of NAS research and highlights recent innovations in the domains of search space and search strategies within NAS for CNN, in Sections III-A and II-B, respectively.

A. Search space

The search space is a critically important element in NAS as it impacts the quality of network design. Baker *et al.* [25] connected various operations in a chain structure, including convolution and pooling, to construct a network. Furthermore, the maximum number of layers in the network and certain hyperparameters, such as the number of convolutional kernels, kernel size, and step size, are also subject to

exploration. However, when there are too many layers, the gradient tends to vanish, limiting the performance of single-chain networks. Real *et al.* [26] introduced skip connections into the search space, creating a multi-branch search space that enables subsequent layers to freely receive the output of previous layers, thus enhancing diversity in the search space. While the chain structure-based search space exhibits strong representation capabilities in architecture search by initialising network architecture randomly, it can contain some impractical network structures. As a result, exhaustively covering the search space can be computationally costly.

Zoph *et al.* [27] proposed a cell-based search space that includes two types of cells: normal cells, retaining input dimensions, and reduced cells, reducing spatial dimensions. The final network is composed of stacked cells, each cell containing fewer layers, making the search process more efficient. However, determining how to connect cells and design cell structures poses its own set of challenges. Zoph *et al.* [27] employed sequential cells for constructing the network structure, with each cell taking the outputs of two preceding cells as input. Cai *et al.* [28] closely linked each cell to the next cells, while Liu *et al.* [16] designed cells with multiple operations between nodes and explored promising cells by selecting connection modes. The space defined in [16] appears to be a good compromise as it is a searchable space that is not excessively large and yet allows the identification of architectures with high quality. It may be observed that some studies highlight the difficulties in detecting optimal solutions within the vast search space. As a countermeasure to this issue, Guo *et al.* [29] proposed the adoption of an evolved search space to progressively approach the optimal solution.

In this study, for the reasons mentioned above, we utilise the cell search space proposed in [16].

B. Search strategy

Early NAS approaches were conducted using reinforcement learning, which treats neural architecture generation as an action taken by an agent. The agent's rewards are determined based on the evaluation performance of the trained architecture. Representing the agent's policy and optimising it are crucial steps in this process. Baker *et al.* [25] proposed a Q-learning training strategy to select operations and hyperparameters. Zoph *et al.* [15] sequentially sampled strings and encoded network structures using RNN strategies. However, these methods required training hundreds or even thousands of candidate network architectures, resulting in significant computational overhead.

Subsequently, several algorithms have been designed to address this issue. Chen *et al.* [30] introduced a neural architecture comparator to provide comparative results to the architecture generator, which, in turn, utilises these results to produce sampled architectures and update the baseline architecture. Guo *et al.* [31] proposed an approach based on operation transitions, employing a Markov decision process

and expanding the transformation space to obtain network architectures with enhanced performance. Pan *et al.* [32] introduced a fitness approximation-based evolutionary search algorithm, employing a predictive model to estimate the fitness of the population. Individuals with high fitness and diversity are selected to provide accurate results, which are then used to train the predictive model. These methods employ additional models to assist in evaluating the quality of the searched architectures.

To significantly improve search speed, a gradient-based method introduces structural weights, i.e. DARTS [16] making the search space continuous and optimising network parameters and structural weights using gradient descent was introduced. Numerous improved algorithms have enhanced DARTS from various perspectives. β -DARTS [33] applied a novel and generic Beta-decay regularisation loss to enhance the robustness and effectiveness of DARTS. Shapley-NAS [34] utilises the concept of marginal contribution to assess the significance of candidate operations and demonstrates that, when compared to the α values in DARTS, Shapley values are more efficient in selecting the most representative operations. However, due to the strong coupling between operations in the search space (coupling between operation weights and presence of the corresponding operation in the candidate subnetwork), the structure often falls into local optima due to the preference for weight-free operations when searching for neural architectures, resulting in a single, poorly-performing generated structure.

Evolutionary computation methods possess strong global search capabilities and better explore the search space through operations like crossover and mutation. Genetic CNN [23] encoded operations such as convolution and pooling to initialise network structures randomly. CNN-GA [35] used genetic algorithms to search for blocks, yielding improved network structures. Nevertheless, these algorithms still incur substantial computational overhead during network structure evaluation.

In this study, to expedite the evaluation process, we employ evolutionary computation to initialise structural weights of the supernet and generate subnet individuals to enhance global search capabilities. We also use gradient-based search to refine the solutions detected by the evolutionary algorithm. Furthermore, the global search operates within a decoupled space, thereby mitigating the coupling problem that arises during local search.

III. GRADIENT-GUIDED EVOLUTIONARY NEURAL ARCHITECTURE SEARCH

This section describes the algorithmic details of the proposed gradient-guided evolutionary neural architecture search (GENAS) method. Section III-A defines the search space and encoding used by GENAS. Section III-B introduces the objective function used during the NAS process. Section III-C introduces the evolutionary operators utilised by the GENAS framework. Section III-D describes the functioning of the local search operator. Finally, the functioning of the entire GENAS algorithm is also presented.

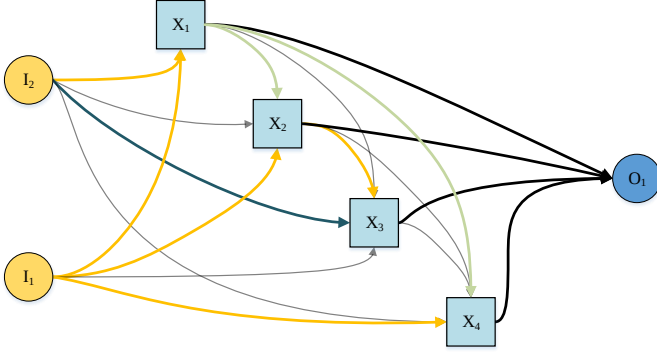


Fig. 1: Structure of a generic cell containing $M = 4$ intermediate nodes.

A. Search Space and Encoding/Decoding of the Subnet Individuals

In this study, to reduce the consumption of computational resources when retraining each subnet individual, we employ a search space inspired by DARTS [16]. Specifically, the search space is defined by the supernet SN , which consists of both normal and reduction cells. Reduction cells can halve the image size and double the number of feature maps. Each candidate architecture is represented as a subnet SN_i of the supernet SN . The parameters of the supernet SN , such as the parameters of the convolution operations, are collectively indicated by ω , while the parameters of the subnet SN_i are indicated by ω_i . Within this structure, each subnet SN_i (just like the supernet SN) is composed of multiple interconnected cells.

The cell is a directed graph consisting of two input nodes I_1 and I_2 , M intermediate nodes x_i with $i = 1, 2, \dots, M$, and one output node O_1 , as illustrated in Fig. 1. Each input node is connected to all the intermediate nodes, and all the intermediate nodes are connected to the output node. Conversely, a connection between two intermediate nodes x_i and x_j is allowed only when $j > i$. In the rest of this section, we may omit the input and output nodes in figures to simplify the graphical representation of the cells.

Each intermediate node signifies a feature map transformed by the graph edges. The edges connecting each pair of nodes x_i and x_j encompass a pool O of eight potential operations:

$$O = \{Zero, SkipConnection, AvgPooling, MaxPooling, Sep3x3Conv, Sep5x5Conv, Dil3x3Conv, Dil5x5Conv\}$$

where

- *Zero* [16] means that there is no connection and thus no feature map flow between two nodes.
- *SkipConnection* [7] means that the feature map in the starting node is copied in the arrival node and added to the feature map at the arrival node. Thus, *SkipConnection* unlike *Zero* allows the feature map

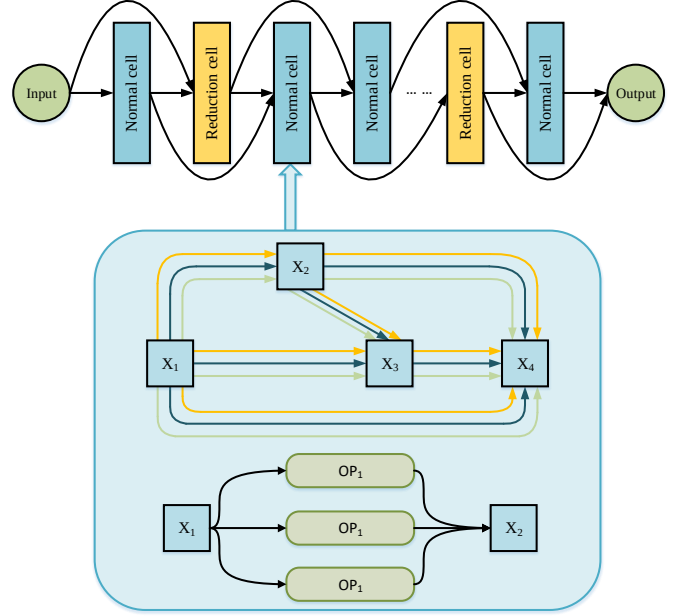


Fig. 2: The structure of the supernet and an example of a cell. The supernet consists of normal and reduction cells. Each cell is a directed graph, with nodes x_i representing feature maps and edges representing the possible operations (indicated by the multiple arrows of various colours between nodes) used to transform one feature map into another.

flow but, unlike the operations below, does not manipulate this feature map.

- *AvgPooling* means that the average pooling operation with kernel size of 3×3 is performed to the feature map.
- *MaxPooling* means that the max pooling operation with kernel size of 3×3 is performed to the feature map.
- *Sep3x3Conv* [36] means that the separable convolution operation with 3×3 kernel is performed to the feature map.
- *Sep5x5Conv* [36] means that the separable convolution operation with 5×5 kernel is performed to the feature map.
- *Dil3x3Conv* [37] means that the dilated convolution operation with 3×3 kernel is performed to the feature map.
- *Dil5x5Conv* [37] means that the dilated convolution operation with 5×5 kernel is performed to the feature map.

Fig. 2 depicts the structure of the supernet and one cell (we display only the intermediate nodes) and the details between a pair of nodes.

For each edge (i, j) , each operation $k \in O$ is associated with an operation weight parameter $\alpha_k^{(i,j)'}.$

These operation weight parameters are normalised according to Eq. (1), as described in [16]:

$$\alpha_k^{(i,j)} = \frac{\exp(\alpha_k^{(i,j)'})}{\sum_{k \in O} \exp(\alpha_k^{(i,j)'})}. \quad (1)$$

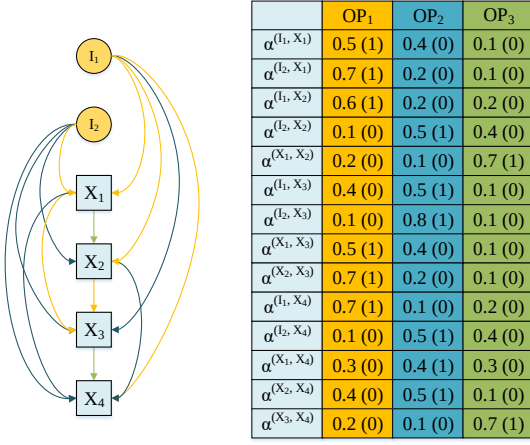


Fig. 3: Representation of a candidate architecture using the proposed tabular encoding of a cell: the allowed connections are listed in the first column, while the other coloured columns indicate the type of operation. Each table entry indicates, in parentheses, the presence (1) or absence (0) of a connection and the value of the corresponding $\alpha_k^{(i,j)}$.

To represent a cell, we require the normalised operation weight parameters $\alpha_k^{(i,j)}$ and information on whether or not node x_i is connected to node x_j through operation k . We propose using a tabular encoding in which allowed connections are listed by rows, with each column representing one type of operation. Each table entry contains two numbers: the parameter $\alpha_k^{(i,j)}$ and a binary value $b_k^{(i,j)}$ that is 0 to indicate the absence of a connection and 1 to indicate a connection between nodes. Importantly, each row can have only one (1) value, meaning that only one operation is allowed between each pair of nodes in the final architecture design. To illustrate our representation and the search space, Fig. 3 displays the proposed tabular encoding of a cell consisting of four intermediate nodes, with three possible operations highlighted in three different colours for convenience. The representations of normal and reduction cells follow a similar pattern. Since a cell serves as the building block of a subnet (a subnet comprises a sequence of multiple normal and reduction cells of the same type), the design of one normal and one reduction cell uniquely encodes a candidate architecture.

In this representation, the NAS problem for cells of M intermediate nodes involves determining $\frac{M^2+3M}{2}$ ($\frac{M^2-M}{2}$ for the edges between intermediate nodes plus $2M$ for the edges between input nodes and intermediate nodes) real numbers between 0 and 1, representing the $\alpha_k^{(i,j)}$ values, and $\frac{M^2+3M}{2}$ binary numbers. Consequently, the search space S is given by:

$$S = [0, 1]^{\frac{M^2+3M}{2}} \cup \{0, 1\}^{\frac{M^2+3M}{2}}$$

where $\sum_{k \in O} b_k^{(i,j)} = 1$ for all edges (i, j) .

The edges connecting the output node to the intermediate nodes are omitted since they do not contain operations but

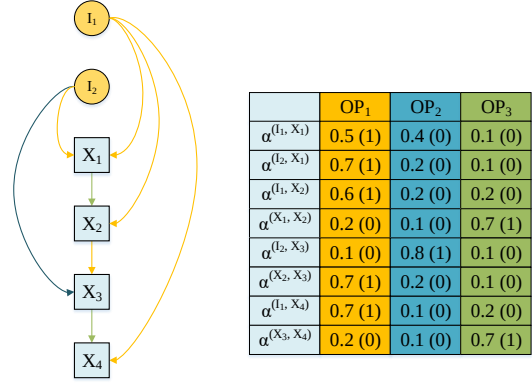


Fig. 4: Representation of a candidate architecture using the proposed tabular encoding after pruning. The original architecture in Fig. 3 contains 14 edges, while the pruned one contains 8 edges with two incoming edges in each intermediate node.

only copy the calculated feature maps.

B. Objective Function and Optimisation Problem

To evaluate the quality of a candidate architecture, as illustrated in Fig. 3, we first perform one of the hard pruning rules used in DARTS [16]: we retain the two incoming edges with the largest $\alpha_k^{(i,j)}$ values while we prune all the others. Fig. 4 displays an example of a solution following the pruning.

With the pruned architecture, we multiply each $\alpha_k^{(i,j)}$ by its corresponding $b_k^{(i,j)}$, resulting in a vector of $2M$ elements (from the $\frac{M^2+3M}{2}$ elements of the original one). Let α denote the vector containing the architectural parameters of the entire subnet and let $\mathcal{L}_{train}$ and $\mathcal{L}_{val}$ represent the Cross-Entropy loss on the training and validation sets. The quality of a candidate solution is assessed by:

$$\mathcal{L}_{val}(\omega^*(\alpha), \alpha) \quad (2)$$

where

$$\omega^*(\alpha) = \arg \min_{\omega} \mathcal{L}_{train}(\omega, \alpha).$$

The minimisation of the objective function in Eq. (2) with respect to α is the NAS problem addressed in this study.

C. Initialization and Evolutionary Operators

At the beginning of the optimisation process, a population of N subnets is randomly sampled from the supernet and encoded according to the tabular encoding depicted in Fig. 3. Unlike many ENAS methods that initialise invalid networks, the search space of GENAS guarantees that the entire population is composed of meaningful architectures. The initialised architectures inherit the weights ω_i from the supernet. The quality of the candidate architectures is then assessed using the procedure in Section III-B.

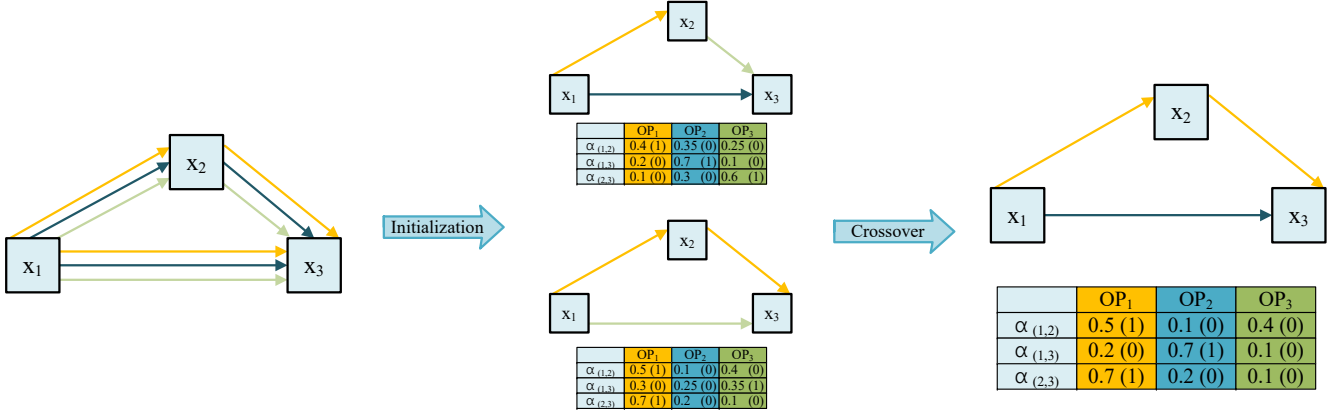


Fig. 5: Initialization and Crossover: Subnets are sampled from the supernet and then undergo crossover. The offspring solution is a combination of the rows of its two parents. In this case, the offspring inherits the first and third rows from one parent and the second row from the other parent.

In each generation, the calculated objective function values are used to select a pair of parent candidate architectures through tournament selection with a tournament size T . These two architectures undergo crossover with a crossover rate t_c , and then the offspring architectures are generated, inheriting one row of the table from either one parent or the other. Each row represents an allowed edge of the subnet. Fig. 5 depicts the initialisation and crossover operation in a simplified case showing only three intermediate nodes. It can be observed that the offspring architecture inherits the first and third rows from one parent and the second row from the other parent. For simplicity, we omitted the depiction of the second offspring solution in Fig. 5, as it would be the dual of the one already shown. The newly generated architecture then undergoes mutation. With a mutation rate t_m , the mutation operator re-samples one row of the individual. Fig. 6 exemplifies the mutation process. In the case depicted in Fig. 6, the third row is selected, and its values are re-sampled, ensuring that only one (1) appears in the mutated row. The generated candidate architectures inherit the weights ω_i from the supernet and their performance assessed using the procedure in Section III-B. The offspring population replaces the population from the previous generation. However, the elite (best) individual

ever found, denoted as p_{best} , is retained at each generation.

For the sake of clarity, Algorithm 1 details the working principles of the implemented crossover and mutation.

D. DARTS Local Search

Every n generations, the DARTS local search is applied to the entire population P . For a given candidate architecture in its tabular encoding, as shown in Fig. 3, initially, all the $b_k^{(i,j)}$ values are set to 1. Then, let us indicate with α_D a vector of length $\frac{M^2+3M}{2} \times 8$ containing all the $\alpha_k^{(i,j)}$ values of a cell. The gradient descent is then applied to α_D to minimise the objective function in Eq. (2). In this scenario as well, each enhanced architecture inherits its corresponding weights ω_i from the supernet before undergoing quality assessment.

The output of the local search is then post-processed by the second hard pruning operations to obtain a meaningful architecture coherent with the tabular encoding proposed in this study. The hard pruning operation used in this case is as follows. For each group of eight $\alpha_k^{(i,j)}$ values associated with one edge, the highest is retained, and the others are discarded. This operation means that an operation $\hat{k}$ is selected, and $b_k^{(i,j)}$ is set to 1, while the values of $b_k^{(i,j)}$ for all the other operations are set to 0. The architectures improved by DARTS are then processed again by the evolutionary framework for the subsequent n generations.

During the local search, all selections are made based on the $\frac{M^2+3M}{2} \times 8$ -dimensional vector α_D , where hypothetical hybrid operations are assumed between nodes. This mechanism, while allowing the search for the optimal set of operations, is also the cause of the coupling problems in DARTS, as the multiple candidate operations coexisting between each pair of nodes would interfere with each other and the consequent pruning would arbitrarily couple the value of the operation weight with the presence of the corresponding edge [19]. GENAS partly addresses this issue by manipulating candidate architectures in their decoupled form, as shown in Fig. 3, during the evolutionary search.

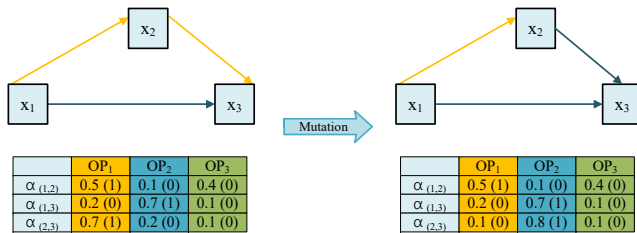


Fig. 6: Mutation: The third row is randomly selected, and its values are re-sampled, ensuring that only one (1) appears in the mutated row.

Algorithm 1 Crossover and mutation

Input: The Population P , crossover rate t_c , mutation rate t_m .

Output: The offspring population P_t .

```

1:  $P_c \leftarrow \emptyset$ 
2: Select the best individual  $p_{best}$  in population  $P$ .
3:  $P_c \leftarrow P_c \cup p_{best}$ 
4: while  $|P_c| < |P|$  do
5:    $p_1 \leftarrow$  Select  $T$  individuals from population  $P$  randomly and choose the best one.
6:    $p_2 \leftarrow$  Repeat Line 5.
7:   for each row  $l_i$  in the tables  $p_1, p_2$  do
8:     Generate a random number  $t_1$  from (0,1).
9:     if  $t_1 > t_c$  then
10:      Assign the weight of row  $l_i$  in  $p_1$  to the new individual.
11:     else
12:      Assign the weight of row  $l_i$  in  $p_2$  to the new individual.
13:     end if
14:   end for
15: end while
16: for  $p_i$  in  $P_c$  do
17:   for  $l_j$  in  $p_i$  do
18:     Generate a random number  $t_2$  from (0,1).
19:     if  $t_2 > t_m$  then
20:       Randomly generate a new set of weights for row  $l_j$ .
21:     end if
22:   end for
23:    $P_t \leftarrow P_t \cup p_i$ 
24: end for
25: return The offspring population  $P_t$ .
```

The evolutionary operators, unlike DARTS, do not couple the values of the operation weights to the presence of the corresponding operation in the candidate architecture. Conversely, the evolutionary operators of GENAS process actual candidate architectures that allow only one operation between nodes and test their performance (directly linked to the model accuracy).

At the conclusion of the optimisation process, the best solution, denoted as p , undergoes the pruning procedure described in Section III-B, getting the final cell design. For the sake of clarity, Algorithm 2 displays the overall framework of GENAS.

IV. EXPERIMENTS AND RESULT ANALYSIS

In the experiments, GENAS is utilised to explore the search space on CIFAR-10/100. Initially, cells are searched on CIFAR-10/100, and the best cells obtained are assembled to create new networks to assess their classification accuracies. Then the architectures searched on CIFAR-10/100 are transferred to ImageNet for performance evaluation. Finally, GENAS is compared with other algorithms in regard to

Algorithm 2 GENAS framework

Input: Population size N , search generations G_{max} , gradient optimisation interval n .

Output: The best CNN architecture p .

```

1:  $P \leftarrow$  Initialise a population of  $N$  subnet individuals.
2:  $t \leftarrow 0$ 
3: while  $t < G_{max}$  do
4:   Encode each subnet by means of the tabular encoding and train the supernet.
5:   if  $t \% n == 0$  then
6:     Represent each candidate architecture as a vector of operation weight parameters  $\alpha_D$ . Apply gradient-descent to all the candidate architectures to locally optimise them.
7:   end if
8:    $P_t \leftarrow$  Evolve population  $P$  by applying evolutionary operators.
9:   Evaluate the fitness of each individual in population  $P_t$ .
10:  Add the elite  $p_{best}$  to the population if not yet included.
11: end while
12: return The best individual  $p$  in population  $P_t$ .
```

search time, the number of network model parameters, and classification accuracy to demonstrate its effectiveness.

A. Datasets

The number of images in CIFAR-10 and CIFAR-100 is 60,000, with each image having a resolution of 32×32 pixels. CIFAR-10 comprises 10 classes, each containing 6,000 images. CIFAR-100 consists of 100 classes, each comprising 600 images. The ImageNet training set includes 1,280,000 color images, resized to a resolution of 224×224 pixels for fair comparison. ImageNet is categorized into 1,000 classes, each having between 732 and 1,300 images. The ImageNet test set comprises 50,000 images, with 50 images for each category.

B. Baseline Algorithms

The twenty-nine compared state-of-the-art (SOTA) algorithms can be categorized into four distinct groups: manually designed architectures, reinforcement learning-based algorithms, gradient descent-based algorithms, and evolutionary computation-based algorithms according to a survey [38].

1) *Manually designed architectures*: We compared our searched architectures with several SOTA CNNs that typically require significant human expertise, including ResNet [7], ShuffleNet [39], DenseNet [40], VGG [6], and MobileNet [41].

2) *Reinforcement learning-based algorithms*: RL-based algorithms, including MetaQNN [25], ENAS [42], NASNet [27], are all SOTA algorithms.

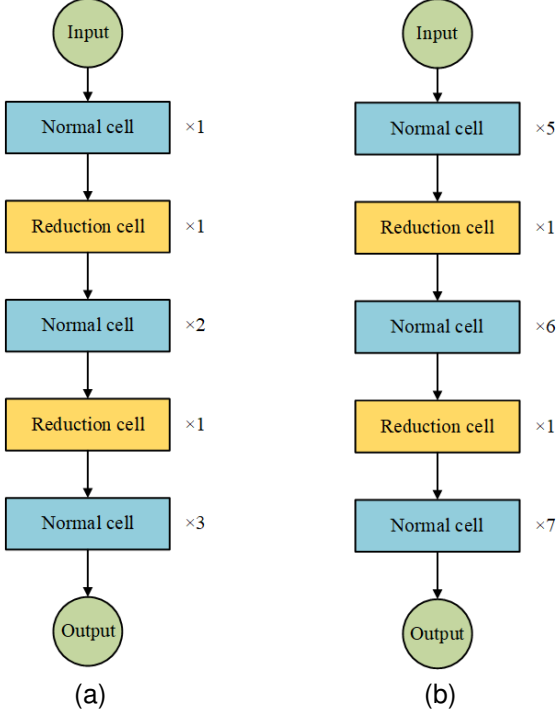


Fig. 7: The architecture of search Fig 7a and test networks Fig. 7b.

3) *Gradient descent-based algorithms*: SMASH [43], GDAS [44], SNAS [45], DARTS [16] are all SOTA algorithms. However, it is noteworthy that the survey predates recent advancements. Notably, recent years have witnessed remarkable progress in the gradient-based NAS field, with SOTA results achieved through enhancements to the DARTS algorithm. These improvements include P-DARTS [46], FairDARTS [47], SDARTS-ADV [48], EoiNAS [49], U-DARTS [50], β -DARTS [33], and Shapley-NAS [34].

4) *Evolutionary computation-based algorithms*: According to the survey, Genetic CNN [23], AmoebaNet [51], Hierarchical Evolution [52], CNN-GA [53], and CARS [54] are all SOTA algorithms. Besides, newly introduced algorithms such as EvNAS [55], MOGIG-Net [56], MOEA-PS [57], MSNAS [24], and NPENAS [58] have also achieved new SOTA levels of performance.

C. Implementation details

1) *Experimental process*: First, eight cells are stacked into a supernet, and the cell structure is searched on the CIFAR-10 and CIFAR-100 datasets, respectively. The searched network is shown in Fig. 7a. After 50 epochs of training, the searched cells are assembled into a new network, i.e., the test network, represented in Fig. 7b. The choice of the test network’s depth has been inspired by [59]. Subsequently, the new network must undergo retraining to evaluate its performance.

2) *Search parameters*: The maximum number of generations, denoted as $G_{\max}$, and batch size are set to 50 and

64 for searching networks. The loss functions, $\mathcal{L}_{\text{train}}$ and $\mathcal{L}_{\text{val}}$, are calculated using cross-entropy loss according to the description in [67]. The network parameters ω are updated using SGD with an initial learning rate of 0.025, a weight decay of 0.0003, and a momentum of 0.9. Additionally, a cosine annealing strategy is applied to reduce the learning rate to 0 over time.

The vector of network architecture parameters, denoted as α_D , is updated using the Adam optimizer with an initial learning rate of 0.0003, a momentum range of (0.5, 0.999), and a weight decay of 0.001. The size of the subnet population is set to 50, with crossover and mutation probabilities of $t_c = 0.5$ and $t_m = 0.1$, respectively. The size of the tournament selection is set to $T = 20$.

D. Experimental results and analysis

1) *Evaluation on the structure of the network*: The network structures searched by GENAS on CIFAR-10/100 are shown in Fig. 8 and Fig. 9, respectively. The network structures have the appropriate number of skip connections. Additionally, the neural architectures searched on CIFAR-100 have more layers than those searched on CIFAR-10. This indicates that the network structures searched on complex datasets have deeper layers, and their deep structure is beneficial for solving complex tasks more effectively. Considering that some of the algorithms used for comparison are multi-objective, i.e., [54], [56], [57], to ensure a fair comparison, we selected from the final population the solution with the highest accuracy value.

2) *Evaluation on CIFAR-10/100*: The network structure, constructed using 20 cells, is trained to evaluate its performance and is shown in Fig. 7b. The training consists of 600 epochs, with a batch size of 96. SGD optimizer, with a 0.9 momentum and a 0.0003 weight decay, is used for training network parameters. The initial learning rate of 0.025 is decreased to 0 using a cosine annealing strategy. To prevent overfitting during the training process, the parameters for cutout length, auxiliary weight, and drop path probability are set to 16, 0.4, and 0.3, respectively.

In Table I, we compare the experimental results of GENAS with some manually designed networks, RL, gradient-based, and evolutionary methods on CIFAR-10/100. GENAS achieves test errors of 2.45% and 16.86% on CIFAR-10/100. The search cost is 0.26 GPU days, making GENAS faster than RL, gradient-based, and evolutionary methods. When compared to GDAS [44], which has the lowest time overhead among all the listed algorithms, we achieve better classification accuracy in a similar search time. In comparison to DARTS, GENAS reduces the classification error by 0.31% and 0.68% on CIFAR-10/100 with a much faster search speed. Furthermore, in terms of model parameters, GENAS achieves better classification results with only 3.53 million parameters. As shown in Table I, GENAS has achieved a smaller mean value and standard deviation than its competitors in most of the cases under scrutiny.

TABLE I: Classification test errors of GENAS on CIFAR-10/100.

Architecture	Params(M)	Search Overhead(GPU days)	Test Accuracy		Search Strategy
			CIFAR-10(%)	CIFAR-100(%)	
ResNet(depth=110) [7]	1.7	-	6.61±0.06	25.16	Manual
ShuffleNet [39]	1.06	-	9.13	22.86	Manual
DenseNet-BC [40]	25.6	-	3.46	17.18	Manual
VGG [6]	20.1	-	6.66	28.05	Manual
MobileNetV2 [41]	2.2	-	4.26	19.20	Manual
MetaQNN [25]	-	100	6.92	17.14	RL
ENAS [42]	4.6	0.5	2.89	19.43	RL
NASNet-A [27]	3.3	1800	2.65	16.82	RL
SMASH [43]	16	1.5	4.03	-	gradient
GDAS [44]	3.4	0.2	2.93	19.38	gradient
SNAS [45]	2.8	1.5	2.85	17.55	gradient
DARTS (first order) [16]	3.3	1.5	3.00	17.76	gradient
DARTS (second order) [16]	3.3	4.0	2.76	17.54	gradient
P-DARTS [46]	3.4	0.3	2.50	17.20	gradient
FairDARTS [47]	2.8	0.4	2.54	17.61	gradient
SDARTS-ADV [48]	3.3	1.3	2.61	16.73	gradient
EoiNAS [49]	3.4	0.6	2.50	17.30	gradient
U-DARTS [50]	3.3	4	2.59	16.32	gradient
β -DARTS [33]	3.78	0.4	2.51±0.07	16.52±0.03	gradient
Shapley-NAS [34]	3.4	0.3	2.47	-	gradient
Genetic CNN [23]	-	17	7.10	29.05	evolution
AmoebaNet-B [51]	2.8	3150	2.55	-	evolution
Hierarchical Evolution [52]	15.7	300	3.75	-	evolution
CNN-GA(CIFAR-10) [53]	2.9	35	3.22	-	evolution
CNN-GA(CIFAR-100) [53]	4.1	40	-	20.53	evolution
CARS [54]	3.6	0.4	2.62	-	evolution
EvNAS [55]	3.7±0.08	3.83	2.52±0.04	16.86	evolution
MOGIG-Net [56]	3.0	14	3.13	18.23	evolution
MOEA-PS [57]	3.0	2.6	2.77	-	evolution
MSNAS [24]	3.25	0.23	2.68±0.08	22.8	evolution
NPENAS [58]	3.5	1.8	2.54±0.10	-	evolution
GENAS(avg.)	3.20±0.26	0.26	2.49±0.03	16.96±0.07	evolution+gradient
GENAS(best)	3.53	0.26	2.45	16.86	evolution+gradient

This demonstrates that GENAS attains superior classification performance while also converging faster during training.

3) *Evaluation on ImageNet*: Due to the large scale of input data in ImageNet, the network structure, constructed using 14 cells, is trained to evaluate its performance. The training consists of 250 epochs, with a batch size of 256. The SGD optimiser, with a 0.9 momentum and a 0.0003 weight decay, is applied to train network parameters. The initial learning rate of 0.1 is reduced to 0 using a cosine annealing strategy. To prevent overfitting during the training process, the label smoothing ratio and auxiliary weight parameters are set to 0.1 and 0.4, respectively.

In Table II, we compare the best accuracy of GENAS on ImageNet with some manually designed networks, RL, gradient-based, and evolutionary search methods. GENAS achieves a classification error of 23.9% with 5.0 million parameters on ImageNet. Although the model's parameter count is slightly larger than some other search methods, its classification performance is superior. GENAS reduces the test errors by 2.8% compared to DARTS. This demonstrates that GENAS achieves better classification performance while also converging faster during training.

4) *Additional experiments on statistical significance*: To further strengthen our experimental analysis and following

the indications in [68], we have re-run the proposed GENAS and EvNAS [55] six times on the same GPU machine, i.e., GTX 2080ti, and under the same experimental conditions for CIFAR-10. Table III shows the experimental results where $\#i$ represents the i -th run, and then we show mean values and standard deviations. Based on these results, we have also performed the Wilcoxon test, which shows that the associated p -value is $0.042 < 0.05$, i.e., the GENAS method significantly outperforms the EvNAS method.

E. Ablation studies

To assess the impact of each search operator on GENAS' performance, we conducted experiments by deactivating DARTS local search, crossover, and mutation individually as shown in IV. It is worth noting that disabling any of these search operators leads to a degradation in classification performance. However, turning off DARTS local search also reduces the computational cost of the algorithm, as expected, since DARTS processes highly dimensional vectors ($\frac{M^2+3M}{2} \times 8$ variables). On the other hand, removing crossover or mutation does not seem to significantly affect the search overhead.

Furthermore, it's crucial to highlight that disabling these operators results in NAS designs characterized by a larger

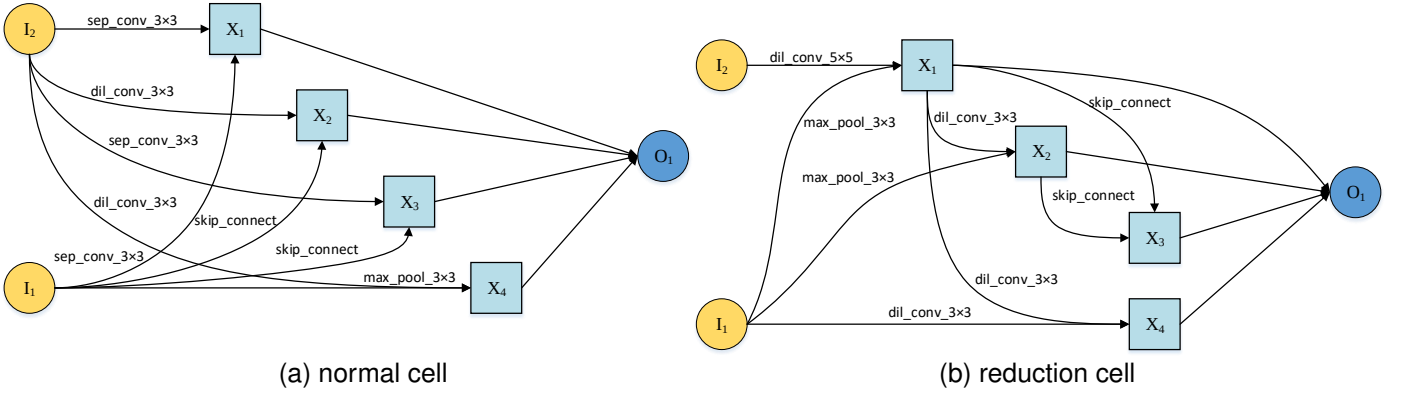


Fig. 8: Structure of cell searched by GENAS on CIFAR-10.

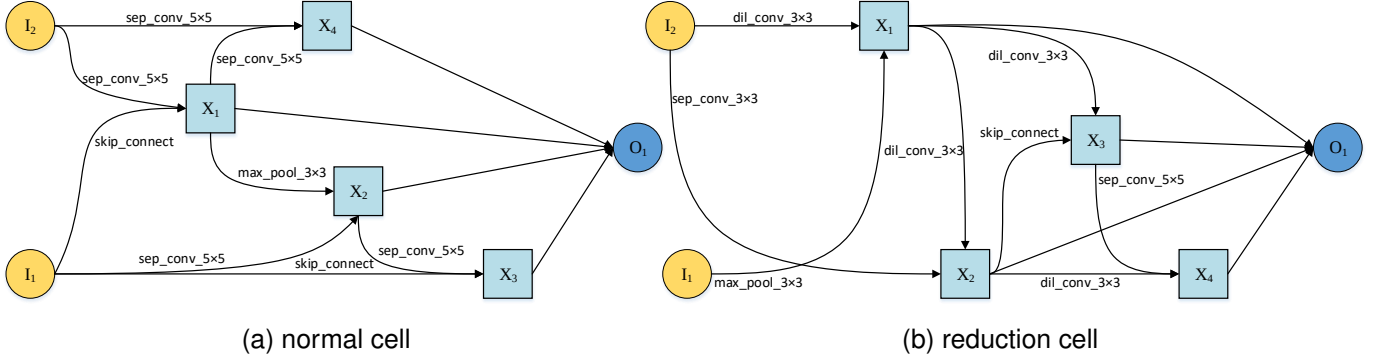


Fig. 9: Structure of cell searched by GENAS on CIFAR-100.

number of parameters. In general, a higher number of parameters implies more convolutional layers, and poor results indicate a lack of rationality in how convolutional combinations are paired or a deficiency in operations like skip-connect, which can impact the model’s convergence. Therefore, we argue that the optimisation of the search process remains incomplete in the absence of any of these modules, leading to architectures that consistently fail to reach their optimal potential.

We conducted an ablation study on the parameter n that triggers the DARTS local search during evolution. We ran the proposed GENAS on the CIFAR-10 dataset with multiple values of n , including 1, 5, 10, 15, and 20. Each experiment was conducted three times, and the corresponding means and standard deviations were recorded. Figure 10 illustrates the network’s accuracy depending on n . It is important to note that the control parameter n governs the balance between the global search conducted by the evolutionary operators and the local search executed by DARTS. This balance is crucial for the successful operation of a memetic algorithm in general and in NAS spaces, as reported in [24]. This ablation study provides strong justification for our parameter settings.

To illustrate the combined impact of global and local search, we highlighted the effects of global evolutionary operators and DARTS local search in Fig. 11. This plot depicts the trend of the validation error over time. Notably, each invocation of DARTS local search takes an average of

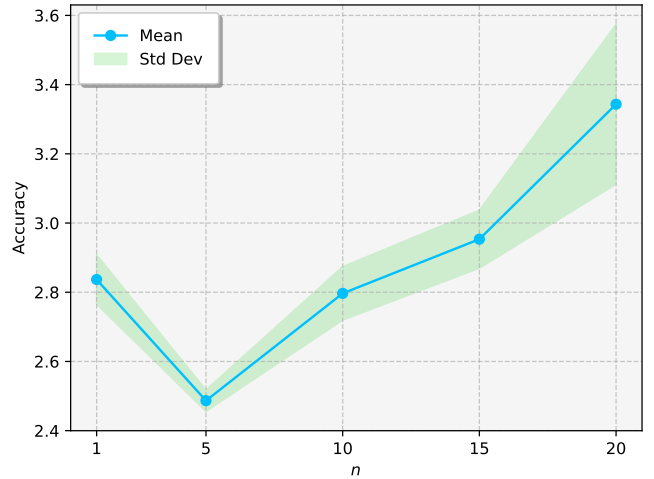


Fig. 10: Ablation study on the frequency of DARTS local search calls. The parameter n indicates the frequency, in terms of generations, triggering DARTS local search.

235 seconds, while $n = 5$ evolutionary generations require 360 seconds. Thus, in GENAS, the budgets allocated for local and global search are approximately 40% and 60% of the total budget, respectively. This budget allocation, along with the results in Table I and Table IV showing the performance of DARTS local search and evolutionary global search standing alone, confirms the efficiency of our parameter setting in Fig. 10 and indicates that our memetic

TABLE II: Classification error rate of GENAS on ImageNet.

Architecture	Params(M)	Search Overhead(GPU days)	Test Errors		Search Strategy
			Top-1(%)	Top-5(%)	
Inception-v1 [60]	6.6	-	30.2	10.1	Manual
MobileNet-v2 [41]	3.4	-	28	9.6	Manual
ShuffleNet-v2 [39]	7.4	-	25.1	9.9	Manual
ResNet50 [7]	25.6	-	24.7	7.8	Manual
NASNet-A [27]	5.3	1800	26	8.4	RL
MnasNet-92 [61]	4.4	1800	25.2	8.0	RL
DARTS [16]	4.7	4	26.7	8.7	gradient
GDAS [44]	5.1	1	26.0	8.5	gradient
SNAS [45]	4.3	1.5	27.3	9.2	gradient
P-DARTS [46]	5.1	0.3	24.7	7.5	gradient
FairDARTS [47]	3.6	0.4	26.3	8.3	gradient
STEN [62]	5.2	1.8	26.7	8.6	gradient
EoiNAS [49]	5.0	0.6	25.6	8.3	gradient
EnTranNAS [63]	4.9	0.06	24.8	7.6	gradient
U-DARTS [50]	4.9	3	26.1	8.1	gradient
β -DARTS [33]	5.4	0.4	24.2	7.1	gradient
Shapley-NAS [34]	5.1	0.3	24.3	-	gradient
AmoebaNet-B [51]	6.4	3150	24.3	7.6	evolution
FairNAS [64]	4.6	12	24.7	-	evolution
CARS [54]	5.1	0.4	24.8	7.5	evolution
SPOS [65]	-	-	25.7	-	evolution
EvNAS [55]	5.1	3.83	24.4	7.4	evolution
SI-EvoNAS [66]	4.7	0.46	24.2	7.4	evolution
MOEA-PS [57]	4.7	-	26.4	8.5	evolution
GENAS	5.0	0.26	23.9	7.2	evolution+gradient

TABLE III: Test errors calculated on multiple runs and statistical significance of the proposed GENAS and EvNAS on CIFAR-10. The associated p -value is 0.042.

Method	#1	#2	#3	#4	#5	#6	mean(std)
EvNAS	2.51	2.47	2.57	2.72	2.75	2.7	2.61±0.11
GENAS	2.47	2.46	2.53	2.57	2.5	2.45	2.50±0.04

TABLE IV: Ablation study on the influence of the search operators on CIFAR-10/100.

Method	Params (M)	Search Overhead (GPU days)	Test Error	
			CIFAR-10	CIFAR-100
GENAS	3.1	0.26	2.47%	17.30%
GENAS w/o DARTS	3.4	0.23	2.71%	17.92%
GENAS w/o crossover	3.5	0.26	2.92%	18.27%
GENAS w/o mutation	3.5	0.26	2.86%	18.21%

framework has a good balance between global and local search.

At last, we tested GENAS’s performance by replacing the DARTS local search, as described, with its improved versions β -DARTS [33] and Shapley-NAS [34]. We named the corresponding GENAS versions β -GENAS and Shapley-GENAS, respectively. The results in Table V show that, although β -DARTS and Shapley-NAS perform better than DARTS (as shown in Table I), their integration into GENAS is detrimental to performance. According to our interpretation, these results can be explained as follows: β -DARTS and Shapley-NAS display better performance than DARTS because both use mechanisms to reduce the unfair competition

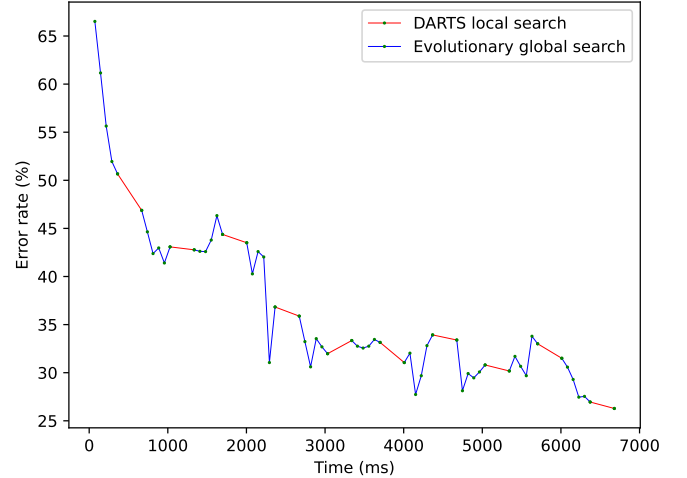


Fig. 11: Validation percentage error vs time during a GENAS run. The blue lines depict the contribution of the evolutionary operators, and the red lines illustrate the contribution of DARTS local search.

among operators embedded in DARTS’ logic. However, in GENAS, the unfair competition among operators is already mitigated by the evolutionary framework as it evolves solutions that allow only one operation for each pair of nodes. Thus, performing a further action on the selection of the operations appears to bias the search towards an excessive use of convolutions. Further investigation is needed to better

understand the architecture design mechanism in case of memetic NAS frameworks.

TABLE V: Ablation study on the DARTS' variant

Algorithm	CIFAR-10(%)	CIFAR-100(%)	Params(M)
β -GENAS	2.76	19.98	3.66
Shapley-GENAS	2.55	17.30	3.21
GENAS(avg.)	2.49 $\pm$ 0.03	16.96 $\pm$ 0.07	3.20 $\pm$ 0.26
GENAS(best)	2.45	16.86	3.53

V. CONCLUSIONS AND FUTURE WORKS

In this paper, we propose GENAS to design effective CNN architectures within a limited computation time. The proposed GENAS is a memetic-inspired hybrid algorithm. GENAS operates on a search space devised by DARTS and utilizes a novel tabular encoding to enable both evolutionary and local search operators to flexibly manipulate candidate architectures.

Since the evolutionary operators can inherit the weights from the supernet, the search time (0.26 GPU days on CIFAR-10/100 for architecture search) is a fraction of that of standard ENAS. The global search operators also appear to support the DARTS local search in quickly detecting promising architectures. Compared to DARTS, the proposed GENAS achieves higher accuracy in a much smaller time. Overall, the novel combination of global and local search, where global search operators are guided by the DARTS space and logic, has been demonstrated to be beneficial, and the role of each search operator seems to be useful, as indicated by the ablation studies. The proposed GENAS has achieved outstanding accuracy results on CIFAR-10/100 when its computational cost is taken into consideration.

Numerical results about the transferability on ImageNet show that the neural model displays the transfer potential of the GENAS design. Our searched architecture outperformed all the other twenty-four transfer models considered in this study in terms of accuracy. It is also competitive with the model designed by FairNAS directly on ImageNet.

Future research will investigate the use of surrogate models based on few shots learning to further reduce computational overhead of GENAS.

REFERENCES

- [1] T. Zhang, S. Jia, X. Cheng, and B. Xu, "Tuning convolutional spiking neural network with biologically plausible reward propagation," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 7621–7631, 2022.
- [2] Y. Liu, S. Liu, Y. Wang, F. Lombardi, and J. Han, "A survey of stochastic computing neural networks for machine learning applications," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 7, pp. 2809–2824, 2021.
- [3] A. V. Savchenko, "Probabilistic neural network with complex exponential activation functions in image recognition," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 31, no. 2, pp. 651–660, 2020.
- [4] M. Zhang, M. Gong, H. He, and S. Zhu, "Symmetric all convolutional neural-network-based unsupervised feature extraction for hyperspectral images classification," *IEEE Transactions on Cybernetics*, vol. 52, no. 5, pp. 2981–2993, 2022.
- [5] K. Muhammad, J. Ahmad, Z. Lv, P. Bellavista, P. Yang, and S. W. Baik, "Efficient Deep CNN-Based Fire Detection and Localization in Video Surveillance Applications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 49, no. 7, pp. 1419–1434, 2019.
- [6] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.
- [8] S. Liu, H. Zhang, and Y. Jin, "A survey on computationally efficient neural architecture search," *Journal of Automation and Intelligence*, vol. 1, no. 1, p. 100002, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2949855422000028>
- [9] S. Li, Y. Sun, G. G. Yen, and M. Zhang, "Automatic design of convolutional neural network architectures under resource constraints," *IEEE Transactions on Neural Networks and Learning Systems*, 2021.
- [10] H. Zhang, Y. Jin, and K. Hao, "Evolutionary search for complete neural network architectures with partial weight sharing," *IEEE Transactions on Evolutionary Computation*, 2022.
- [11] B. Lyu, S. Wen, K. Shi, and T. Huang, "Multiobjective reinforcement learning-based neural architecture search for efficient portrait parsing," *IEEE Transactions on Cybernetics*, pp. 1–12, 2021.
- [12] Z. Lu, G. Sreekumar, E. Goodman, W. Banzhaf, K. Deb, and V. N. Boddeti, "Neural architecture transfer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 9, pp. 2971–2989, 2021.
- [13] Y. Chen, R. Gao, F. Liu, and D. Zhao, "Modulenet: Knowledge-inherited neural architecture search," *IEEE Transactions on Cybernetics*, pp. 1–11, 2021.
- [14] T. Elsken, J. H. Metzen, F. Hutter *et al.*, "Neural architecture search: A survey," *J. Mach. Learn. Res.*, vol. 20, no. 55, pp. 1–21, 2019.
- [15] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," *arXiv preprint arXiv:1611.01578*, 2016.
- [16] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable Architecture Search," in *International Conference on Learning Representations*, 2018.
- [17] T. Zhang, M. Waqas, H. Shen, Z. Liu, X. Zhang, Y. Li, Z. Halim, and S. Chen, "A neural network architecture optimizer based on DARTS and generative adversarial learning," *Information Sciences*, vol. 581, pp. 448–468, 2021.
- [18] H. Wang, R. Yang, D. Huang, and Y. Wang, "iDARTS: Improving DARTS by Node Normalization and Decorrelation Discretization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 4, pp. 1945–1957, 2023.
- [19] Y.-C. Gu, L.-J. Wang, Y. Liu, Y. Yang, Y.-H. Wu, S.-P. Lu, and M.-M. Cheng, "DOTS: Decoupling Operation and Topology in Differentiable Architecture Search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2021, pp. 12 311–12 320.
- [20] S. Xue, R. Wang, B. Zhang, T. Wang, G. Guo, and D. Doermann, "iDARTS: Interactive Differentiable Architecture Search," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 1143–1152.
- [21] Y. Liu, Y. Sun, B. Xue, M. Zhang, G. G. Yen, and K. C. Tan, "A survey on evolutionary neural architecture search," *IEEE Trans. Neural Networks Learn. Syst.*, vol. 34, no. 2, pp. 550–570, 2023. [Online]. Available: <https://doi.org/10.1109/TNNLS.2021.3100554>
- [22] C. He, R. Cheng, C. Zhang, Y. Tian, Q. Chen, and X. Yao, "Evolutionary large-scale multiobjective optimization for ratio error estimation of voltage transformers," *IEEE Transactions on Evolutionary Computation*, vol. 24, no. 5, pp. 868–881, 2020.
- [23] L. Xie and A. Yuille, "Genetic cnn," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 1379–1388.
- [24] J. Dong, B. Hou, L. Feng, H. Tang, K. C. Tan, and Y.-S. Ong, "A Cell-Based Fast Memetic Algorithm for Automated Convolutional Neural Architecture Design," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 9040–9053, 2023.
- [25] B. Baker, O. Gupta, N. Naik, and R. Raskar, "Designing Neural Network Architectures using Reinforcement Learning," in *International Conference on Learning Representations*, 2016.
- [26] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Aging evolution for image classifier architecture search," in *AAAI Conference on Artificial Intelligence*, vol. 3, 2019.

- [27] B. Zoph, V. Vasudevan, J. Shlens, and Q. V. Le, "Learning transferable architectures for scalable image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 8697–8710.
- [28] H. Cai, J. Yang, W. Zhang, S. Han, and Y. Yu, "Path-level network transformation for efficient architecture search," in *International Conference on Machine Learning*, 2018, pp. 678–687.
- [29] Y. Guo, Y. Chen, Y. Zheng, P. Zhao, J. Chen, J. Huang, and M. Tan, "Breaking the curse of space explosion: Towards efficient nas with curriculum search," in *International Conference on Machine Learning*. PMLR, 2020, pp. 3822–3831.
- [30] Y. Chen, Y. Guo, Q. Chen, M. Li, W. Zeng, Y. Wang, and M. Tan, "Contrastive Neural Architecture Search with Neural Architecture Comparators," in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2021, pp. 9497–9506.
- [31] Y. Guo, Y. Zheng, M. Tan, Q. Chen, Z. Li, J. Chen, P. Zhao, and J. Huang, "Towards accurate and compact architectures via neural architecture transformer," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 6501–6516, 2021.
- [32] C. Pan and X. Yao, "Neural Architecture Search Based on Evolutionary Algorithms with Fitness Approximation," in *2021 International Joint Conference on Neural Networks (IJCNN)*, 2021, pp. 1–8.
- [33] P. Ye, B. Li, Y. Li, T. Chen, J. Fan, and W. Ouyang, "b-DARTS: Beta-Decay Regularization for Differentiable Architecture Search," in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2022, pp. 10864–10873.
- [34] H. Xiao, Z. Wang, Z. Zhu, J. Zhou, and J. Lu, "Shapley-NAS: Discovering Operation Contribution for Neural Architecture Search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 892–11 901.
- [35] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing cnn architectures using the genetic algorithm for image classification," *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3840–3854, 2020.
- [36] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [37] F. Yu, V. Koltun, and T. Funkhouser, "Dilated Residual Networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2017, pp. 636–644.
- [38] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, X. Chen, and X. Wang, "A comprehensive survey of neural architecture search: Challenges and solutions," *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–34, 2021.
- [39] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," in *European Conference on Computer Vision*. Springer, 2018, pp. 122–138.
- [40] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2017, pp. 2261–2269.
- [41] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [42] H. Pham, M. Guan, B. Zoph, Q. Le, and J. Dean, "Efficient neural architecture search via parameters sharing," in *International Conference on Machine Learning*, 2018, pp. 4095–4104.
- [43] A. Brock, T. Lim, J. Ritchie, and N. Weston, "SMASH: One-Shot Model Architecture Search through HyperNetworks," in *International Conference on Learning Representations*, 2018.
- [44] X. Dong and Y. Yang, "Searching for a robust neural architecture in four gpu hours," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 1761–1770.
- [45] S. Xie, H. Zheng, C. Liu, and L. Lin, "SNAS: stochastic neural architecture search," in *International Conference on Learning Representations*, 2018.
- [46] X. Chen, L. Xie, J. Wu, and Q. Tian, "Progressive differentiable architecture search: Bridging the depth gap between search and evaluation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 1294–1303.
- [47] X. Chu, T. Zhou, B. Zhang, and J. Li, "Fair darts: Eliminating unfair advantages in differentiable architecture search," in *European Conference on Computer Vision*, 2020, pp. 465–480.
- [48] X. Chen and C.-J. Hsieh, "Stabilizing differentiable architecture search via perturbation-based regularization," in *International Conference on Machine Learning*, 2020, pp. 1554–1565.
- [49] Y. Zhou, X. Xie, and S.-Y. Kung, "Exploiting Operation Importance for Differentiable Neural Architecture Search," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6235–6248, 2022.
- [50] L. Huang, S. Sun, J. Zeng, W. Wang, W. Pang, and K. Wang, "U-DARTS: Uniform-space differentiable architecture search," *Information Sciences*, vol. 628, pp. 339–349, 2023.
- [51] E. Real, A. Aggarwal, Y. Huang, and Q. V. Le, "Regularized evolution for image classifier architecture search," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 4780–4789.
- [52] H. Liu, K. Simonyan, O. Vinyals, C. Fernando, and K. Kavukcuoglu, "Hierarchical Representations for Efficient Architecture Search," in *International Conference on Learning Representations*, 2018.
- [53] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically Designing CNN Architectures Using the Genetic Algorithm for Image Classification," *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3840–3854, 2020.
- [54] Z. Yang, Y. Wang, X. Chen, B. Shi, C. Xu, C. Xu, Q. Tian, and C. Xu, "CARS: Continuous Evolution for Efficient Neural Architecture Search," in *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE Computer Society, 2020, pp. 1826–1835.
- [55] N. Sinha and K.-W. Chen, "Evolving neural architecture using one shot model," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2021, pp. 910–918.
- [56] Y. Xue, P. Jiang, F. Neri, and J. Liang, "A Multi-Objective Evolutionary Approach Based on Graph-in-Graph for Neural Architecture Search of Convolutional Neural Networks," *International Journal of Neural Systems*, vol. 31, no. 09, p. 2150035, 2021.
- [57] Y. Xue, C. Chen, and A. Slowik, "Neural Architecture Search Based on a Multi-Objective Evolutionary Algorithm With Probability Stack," *IEEE Transactions on Evolutionary Computation*, vol. 27, no. 4, pp. 778–786, 2023.
- [58] C. Wei, C. Niu, Y. Tang, Y. Wang, H. Hu, and J. Liang, "NPE-NAS: Neural Predictor Guided Evolution for Neural Architecture Search," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 11, pp. 8441–8455, 2023.
- [59] Y. Xu, L. Xie, W. Dai, X. Zhang, X. Chen, G.-J. Qi, H. Xiong, and Q. Tian, "Partially-Connected Neural Architecture Search for Reduced Computational Redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 9, pp. 2953–2970, 2021.
- [60] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 1–9.
- [61] M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le, "Mnasnet: Platform-aware neural architecture search for mobile," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2820–2828.
- [62] X. Dong and Y. Yang, "One-shot neural architecture search via self-evaluated template network," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 3681–3690.
- [63] Y. Yang, S. You, H. Li, F. Wang, C. Qian, and Z. Lin, "Towards improving the consistency, efficiency, and flexibility of differentiable neural architecture search," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 6667–6676.
- [64] X. Chu, B. Zhang, and R. Xu, "Fairnas: Rethinking evaluation fairness of weight sharing neural architecture search," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 12 239–12 248.
- [65] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun, "Single path one-shot neural architecture search with uniform sampling," in *European Conference on Computer Vision*, 2020, pp. 544–560.
- [66] H. Zhang, Y. Jin, R. Cheng, and K. Hao, "Efficient evolutionary search of attention convolutional networks via sampled training and node inheritance," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 2, pp. 371–385, 2020.

- [67] Z. Zhang and M. Sabuncu, “Generalized cross entropy loss for training deep neural networks with noisy labels,” *Advances in neural information processing systems*, vol. 31, 2018.
- [68] M. Lindauer and F. Hutter, “Best practices for scientific research on neural architecture search,” *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 9820–9837, 2020.